Reinforcement Learning
Lecture VI: Policy Iteration

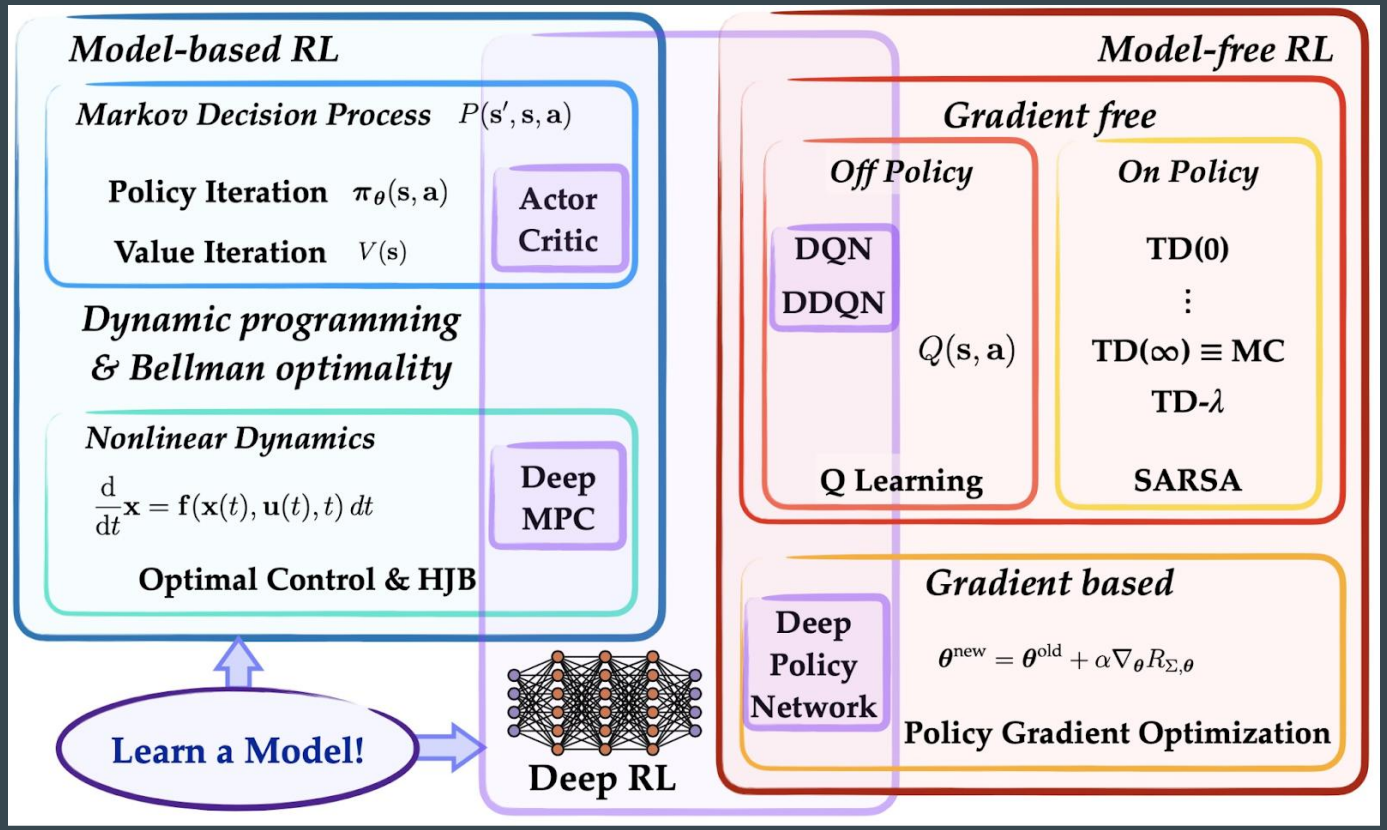
Asad Jeewa
University of Cape Town



What will be covered

- Learning an optimal policy directly

The Bigger Picture



Recall: Model-based RL

- Given: MDP $(S, T, A, R, \boldsymbol{\gamma})$
- Goal: Find an optimal policy: π^*
- How: Exact Methods find the true optimal solution using the known model (T, R).
  - Value iteration
  - **Policy iteration**

Recall: Value Iteration

- Method for finding the optimal value function V^* for an **MDP**.
  - π^* can be derived from V^*.
- Discrete: Assumes a finite state and action space.
- Model-Based: Assumes known transition model T and reward function R
- Dynamic programming (DP) algorithm.

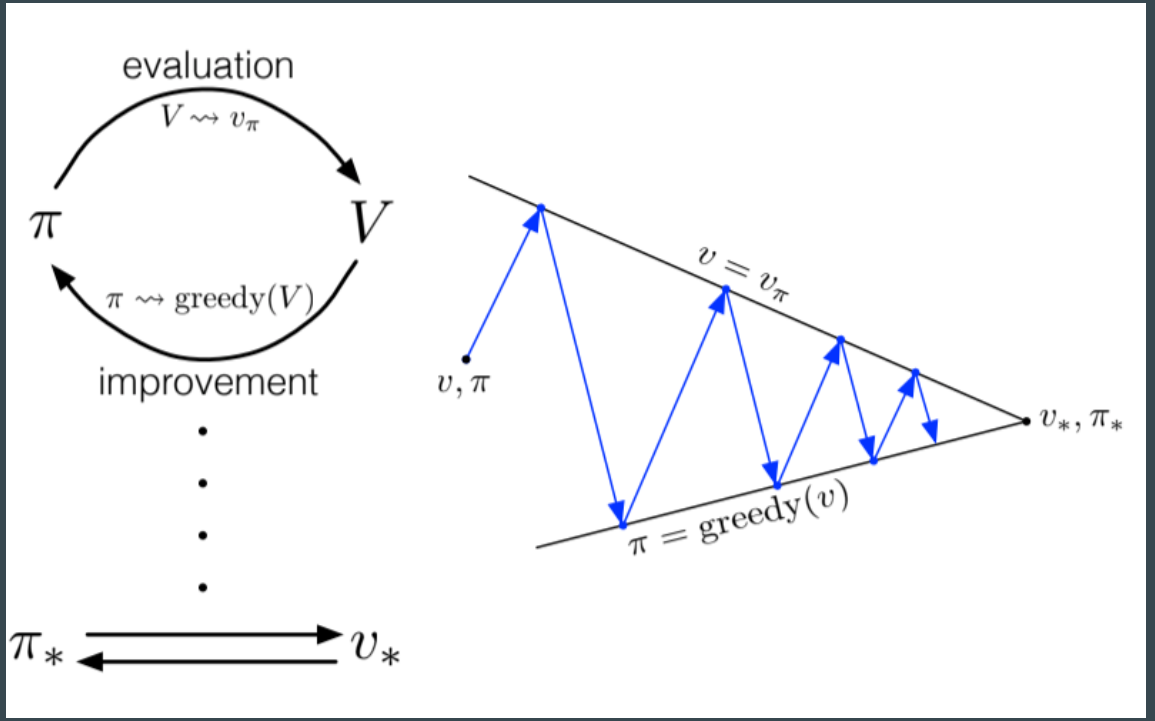
Recall: Dynamic Programming

- Solves problems by combining solutions to smaller, overlapping subproblems.
- In MDPs: A state's value depends on the values of its successor states.
- Bellman equations: Capture this recursive relationship.

Relationship between Policy Iteration and Value Iteration

- Value Iteration
  - Computes the optimal value function V^* directly.
  - Optimal policy π^* derived after convergence from V^*.
- Policy Iteration
  - Searches the space of policies to find π^*.
  - Maintains an explicit policy π.
  - Repeats two steps:
    - Policy Evaluation: compute V^{π_k}.
    - Policy Improvement: update $\pi_k \rightarrow \pi_{k+1}$.

Policy Iteration Visually



- $argmax$ returns the best action for a single state
- $greedy(V)$ constructs a full policy by applying $argmax$ in every state.

Policy Iteration Algorithm

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation
   Loop:
   $\Delta \leftarrow 0$
   Loop for each $s \in \mathcal{S}$:
   $v \leftarrow V(s)$
   $V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s))\big[r + \gamma V(s')\big]$
   $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement
   policy-stable $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
   old-action $\leftarrow \pi(s)$
   $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$
   If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*
   If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

medium.com/@numsmt2/reinforcement-learning-chapter-4-dynamic-programming-part-3-value-iteration-6f01f6347813

Policy Iteration

- Model-Based Algorithm: the transition dynamics T and reward function R are assumed to be known
- Update schemes can be
  - Synchronous: Evaluate all states fully, then improve the policy for all states.
  - Asynchronous: Evaluate and improve individual states one at a time in any order.

Policy Iteration Properties

- Convergence: Guaranteed due to the policy improvement theorum.
- What about the complexity?

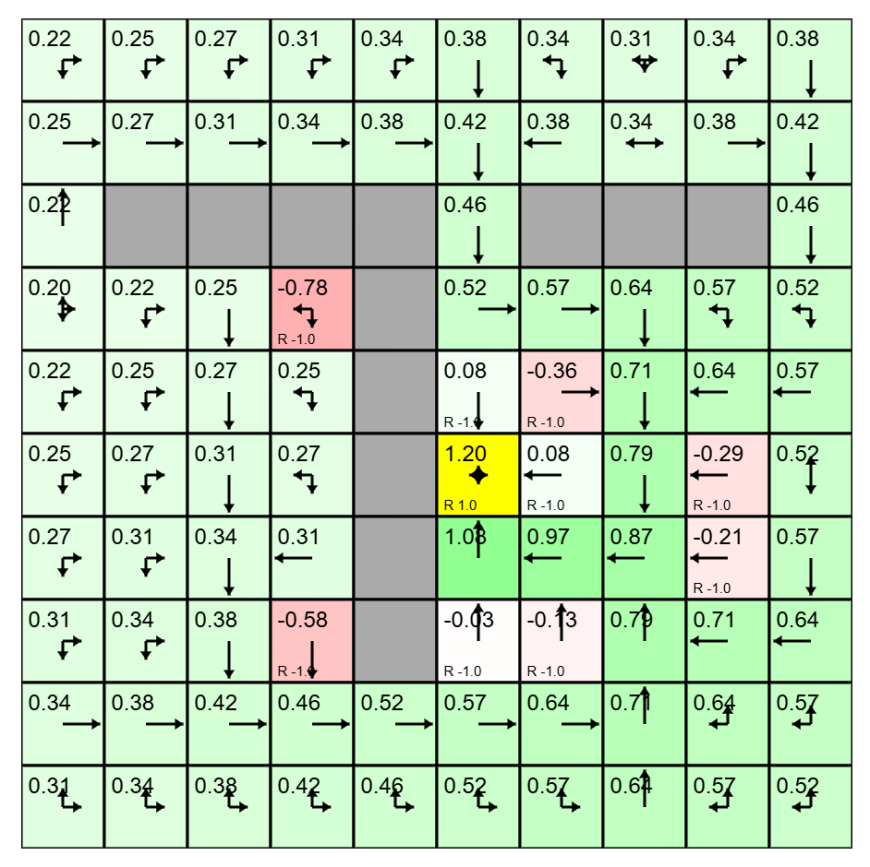
Policy Iteration Algorithm

- Is the algorithm shown synchronous or asynchronous? Why?
- Are we using the Bellman expectation or optimality function?

Policy Iteration Algorithm

- Follows a predictable pattern:
  - Evaluate: "How much is my current plan worth?"
  - Improve: "Can I do better by changing my actions?"
  - Repeat: Keep going until the plan stops changing.
- Guaranteed to find the optimal policy in a finite number of steps.
- Tabular RL: Policy is a lookup table and "improvement" means overwriting the current action with a better one.

Policy Iteration Visually



https://cs.stanford.edu/people/karpathy/reinforcejs/gridworld_dp.html

Which method should I use?

- Policy Iteration
  - Tracks policy improvement explicitly.
  - Fewer iterations to convergence, but each iteration can be costly.
  - Works better if you start with a reasonable initial policy
- Value Iteration
  - Updates values iteratively
  - More iterations to converge but simpler per iteration.
  - Produces optimal policy only after convergence.

Generalized Policy Iteration (GPI)

- Policy evaluation and policy improvement happen together and influence each other, rather than waiting for one to fully finish.
- Policy Evaluation: Can be partial or approximate.
- Policy Improvement: Can be applied after each evaluation step, gradually improving the policy.
- GPI is the framework behind many modern RL algorithms!

PRACTICAL TASK

- The code in the repo has been separated into:
  - Algorithms (includes `value_iteration.py`)
  - Environments (includes `grid_world.py`)
  - Utility functions
- Use the code in the shared space to implement `policy_iteration.py`.
- Tip: Use the value_iteration implementation to guide you. Also refer to the detailed algorithm in the lecture slides for guidance.

Resources

- Shared RL code space
- Sutton & Barto (1998), Reinforcement Learning: An Introduction, Chapter 4.
- https://gibberblot.github.io/rl-notes/single-agent/policy-iteration.html